IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Lead-Time-Scaled Chance Constraints With a Cost-Aware Fallback for Fast Multi-UAV Replanning Under Uncertain Execution

Cao-Phuc Ha[1], Anh Nguyen Vu Minh[1], and Phu-Nguyen Le[2]

[1]Department of Artificial Intelligence, FPT University, Ho Chi Minh City, Vietnam (e-mail: caophucai@gmail.com, anhnvmse203412@fpt.edu.vn)
[2]Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University, Ho Chi Minh City 700000, Vietnam (e-mail: nguyenlp@vaa.edu.vn)

Corresponding author: Phu-Nguyen Le (e-mail: nguyenlp@vaa.edu.vn).

ABSTRACT Multi-UAV replanners usually plan for nominal execution, so under wind, speed and service-time errors, tasks near the end of a route are served late. On development flights, the spread of the finish-time error grew linearly with the lead time, because disturbances persist over a leg. The proposed replanner, LTCC, exploits this structure. With an affine error model, a chance constraint on the finish time of every task reduces exactly to an effective deadline, which fast heuristics reuse unchanged. A cost-aware fallback attempts a task that the constraint gives up only if its probability of meeting the deadline outweighs the cost of a late arrival, again in closed form and without tuning. In six pre-registered studies on held-out missions of a kinematic simulator, the chance-constrained model served more reward than the nominal model and a constant time buffer on three independent sets of seeds, by 5.07 and 4.22 percentage points on the last one, and 10.0% more than the consensus-based bundle algorithm on a further set. A tuned proportional buffer and moment propagation came within 0.38 points. When a late arrival cost one task reward, the cost-aware fallback outscored the nominal fallback by 2.42 points, and a proportional buffer and a safety factor tuned for this cost by 0.90 and 1.78 points (all $p < 0.001$), and it improved moment propagation as well. At twice this cost, the tuned buffer was better. In PX4 software-in-the-loop flights, the fallback raised the utility by 56%, whereas the advantage over the nominal model was not confirmed.

INDEX TERMS Anytime planning, chance-constrained planning, dynamic vehicle routing, execution uncertainty, multi-UAV task allocation, PX4 software-in-the-loop.

I. INTRODUCTION

Teams of unmanned aerial vehicles (UAVs) are used for inspection, delivery and monitoring. During a mission, new tasks may appear, a vehicle may fail, or a deadline may be moved forward. A central planner then repairs the routes of the whole fleet within a fraction of a second, while some UAVs may already wait for new instructions. Dynamic vehicle routing and multi-robot task allocation offer many methods for this repair [1]–[3], and auction methods such as the consensus-based bundle algorithm (CBBA) [4] or metaheuristics such as adaptive large neighbourhood search (ALNS) [5] find good plans within such budgets.

Most of these replanners plan for nominal execution. When UAVs meet wind, fly slower than commanded or take longer at a task, part of the slack that the plan promises is consumed by errors that the planner does not see, and tasks near the end of a route are served late. The error grows with the time until a task is reached, so a constant time margin is too large for early tasks and too small for late ones. Stochastic and robust routing protect schedules against such errors [6]–[8], and chance-constrained allocation bounds the probability of failure [9], [10]. These models are usually solved by sampling or by exact methods that are too slow for the time available between two events. A second line of work treats the computation time of an anytime planner as a decision [11]–[14]. It is not known which of the two, the planning model or the control of computation, matters more for a replanner that faces uncertain execution.

A chance constraint raises a further question. It gives up every task whose probability of being served on time falls below its target, even when a vehicle has nothing better to do, and whether such a task is still worth attempting depends on what a late arrival costs. Soft time windows price lateness in the objective [15], but they need the distribution of the lateness of every route inside the search. A margin without a probability, such as a constant or proportional buffer, cannot

answer the question and has to be tuned again whenever the cost changes.

This paper addresses four research questions. RQ1: how does the finish-time error of a replanned UAV fleet grow along a route, and can a chance constraint built on it run inside fast heuristics? RQ2: which of the planning model, the stopping rule and the trigger matters most? RQ3: how does the resulting replanner compare with baselines from the literature, including time buffers, moment propagation and a published task-allocation algorithm, and does it carry over to PX4 autopilots? RQ4: when a late arrival has a cost, can the probability of meeting the deadline decide which given-up tasks to attempt, without tuning?

To answer them, the paper proposes LTCC (lead-time-scaled chance constraints), a replanner that plans against a chance constraint on the finish time of every task and keeps the fast heuristics of the nominal model. Its central observation is that the probability of meeting a deadline, which a buffer lacks, lets a replanner price lateness without tuning. The contributions are threefold. The first is the planning model. Because wind and airspeed errors persist over a leg, the spread of the error grows linearly with the lead time and not with its square root, as independent increments would imply. With an affine error model, a chance constraint on every task reduces exactly to an effective deadline, which the repair and the search of the nominal model use unchanged (Proposition 1). A cost-aware fallback attempts a given-up task only if its probability of meeting the deadline outweighs the cost of a late arrival. In closed form, this rule is again an effective deadline, and it has no parameter to tune (Proposition 2). The second is a sequence of six pre-registered studies on held-out missions with re-implemented baselines. These include stopping rules from the metareasoning literature [13], [14], [16], fixed and size-scaled budgets, a constant time buffer [7], a proportional buffer, moment propagation along the route [8], CBBA [4] and a conventional event-triggered planner. Competitors are tuned for every cost of a late arrival, and ablations separate the effects of the planning model, the fallback, the stopping rule and the trigger. The third is a software-in-the-loop evaluation with PX4 autopilots [17] in Gazebo [18] and ROS 2 [19], after a rescaling that keeps every dimensionless ratio of the decision problem, with a number of flights computed beforehand from the kinematic tier.

The results are reported as they came out. In the kinematic tier, the chance-constrained model served more reward than the nominal model and a constant time buffer on three independent sets of held-out seeds, while the choice of stopping rule mattered little. Without fallback, it was too cautious under high disturbances with halved windows, and the fallback, designed after this result and tested on new seeds, removed the weakness. A tuned proportional buffer and moment propagation, whose margins also grow with the lead time, reached the same utility within 0.38 percentage points, so the shape of the margin and the fallback carry the gain in utility rather than the precise error model. Re-scoring these runs showed that the nominal fallback pays off only if a late arrival costs less than about one task reward. With a late arrival that costs one task reward, the cost-aware fallback served 2.42 percentage points more than the nominal fallback, and 0.90 and 1.78 points more than a proportional buffer and a safety factor tuned for this cost. At twice this cost, the tuned buffer was better by 1.02 points. On a further set of seeds, the replanner served 4.86 points, or 10.0%, more utility than CBBA. With a late cost of one task reward, CBBA itself gained 12.12 points when it planned on the proposed model, and the cost-aware fallback improved moment propagation as well. In software-in-the-loop flights with PX4 and Gazebo, the fallback raised the utility by 10.35 percentage points, whereas the advantage over the nominal model was not confirmed, as the kinematic tier had predicted for the same missions. An earlier pre-registered study, in which only the stopping rule was made execution-aware, found no significant gain in utility (Section VI-A).

II. RELATED WORK

A. DYNAMIC TASK ALLOCATION AND ROUTING

Multi-robot task allocation [1] and dynamic vehicle routing [2], [3] reassign tasks when information changes. The planning problem considered here is a prize-collecting routing problem with time windows [20], [21]. Large neighbourhood search has been adapted to dynamic routing with time windows [22]. For multi-UAV systems, auction methods such as CBBA reach conflict-free assignments quickly [4]. Recent work handles stochastic task completion with hierarchical policies [23], uncertain execution times with dynamic time slices [24], intermittent communication with uncertainty-gated distributed allocation [25], and events with assignment-preserving or local repair [26], [27]. Event-triggered and periodic allocation have been compared for multi-UAV systems [28], in line with the rescheduling literature [29], [30]. Calvo and Capitan validate a repair-based architecture with PX4 and Gazebo [31], and Ali et al. use ArduPilot software-in-the-loop simulation [32]. Table 1 compares these approaches.

B. UNCERTAINTY IN ROUTING AND ALLOCATION

Stochastic vehicle routing models travel and service times as random variables [35]. With hard time windows and stochastic service times, a priori routes with recourse [6] and robust formulations over budgeted uncertainty sets [7] protect the schedule against delays, at the price of a harder model. Scenario-based approaches plan against sampled futures [36], and chance-constrained task allocation bounds the probability that a team misses its objectives [9]. Per-customer chance constraints on the service time windows were formulated for routing with stochastic travel and service times by Li et al. [37]. Ehmke et al. [8] propagate the mean and variance of the arrival time along a route, approximate the waiting for a time window by Clark's moments of a maximum [38], and use the result as a feasibility check inside routing heuristics. Soft time windows take the opposite route and charge a penalty for every late service inside the objective. Taş et al. [15] minimise transportation costs plus expected early and late service costs

TABLE 1. Features of related approaches to task allocation, routing and replanning under uncertainty compared with the proposed method. TW: time windows or deadlines; Unc.: execution or duration uncertainty in the planning model; Late cost: a cost of late service that enters the planning decision; Online: replanning during the mission; Trigger: replanning trigger policies compared; Stop: when to stop the replanner (computation as a decision); SITL: validation with an autopilot in software-in-the-loop or physics simulation. ✓: yes; ○: partly; –: no or not reported.

Work	Problem	TW	Unc.	Late cost	Online	Trigger	Stop	SITL	Solver or architecture
Taş et al. [15]	VRP, soft time windows	✓	✓	✓	–	–	–	–	tabu search
Ehmke et al. [8]	VRPTW, service levels	✓	✓	–	–	–	–	–	heuristic, moments
Zhou and Tokekar [33]	multi-robot assignment	–	✓	–	–	–	–	–	greedy, CVaR
Gao et al. [10]	UAV assignment	✓	✓	–	–	–	–	–	MILP, DRCC
Wang et al. [34]	UAV task planning	✓	✓	–	–	–	–	–	completion-time estimate
Choudhury et al. [23]	MRTA, time windows	✓	✓	–	✓	–	–	–	hierarchical
Huang et al. [24]	multi-UAV allocation	–	✓	–	✓	○	–	–	–
Calvo and Capitan [31]	long-endurance UAV MRTA	○	–	–	✓	–	–	✓	centralised
Hwang et al. [26]	multi-UAV replanning	✓	–	–	✓	–	–	–	–
Shen et al. [27]	UAV swarm allocation	○	–	–	✓	–	–	–	edge
Qian and Fu [25]	multi-UAV allocation	✓	✓	–	✓	–	–	–	distributed
Cong et al. [28]	multi-UAV allocation	✓	○	–	✓	✓	–	○	–
Ali et al. [32]	multi-UAV allocation	–	–	–	✓	–	–	✓	decentralised
Svegliato et al. [14]	anytime metareasoning	–	–	–	–	–	✓	–	–
Proposed	multi-UAV replanning	✓	✓	✓	✓	✓	✓	✓	centralised

under stochastic travel times with tabu search, which requires the distribution of the lateness of every route during the search. For multi-UAV systems, Gao et al. [10] formulate a distributionally robust chance-constrained task assignment with time windows under uncertain fuel consumption and solve a mixed-integer reformulation, Wang et al. [34] estimate the effect of uncertain flight times on completion times and time-window violations without Monte Carlo simulation, and Zhou and Tokekar [33] select risk-averse multi-robot assignments through the conditional value at risk. These formulations rely on mixed-integer, sampling-based or greedy solvers over a planning horizon, and none of them is evaluated as a replanner that has to publish a plan within a fraction of a second after every event.

The proposed model differs in four respects. Its error model is fitted to the finish times observed in closed loop rather than assumed per arc. It is applied in a replanner that runs between events within a fraction of a second, with a publication delay. It adds a fallback for the tasks that a hard chance constraint gives up. And it prices a late arrival only in this fallback, through a break-even probability that again yields a deterministic deadline, so the search keeps its objective and its fast feasibility checks, whereas soft time windows change both. Moment propagation, a proportional buffer, a constant buffer and the consensus-based bundle algorithm (CBBA) [4] are re-implemented as baselines.

C. METAREASONING AND ANYTIME ALGORITHMS

Anytime algorithms [11], [13] and the rational allocation of computation [12], [16] decide how long a planner should compute, from performance profiles [39], online performance prediction [14] or learned stopping policies [40]. Situated temporal planning and deadline-aware search account for the passage of time during planning [41]–[43]. Two of these rules are re-implemented as baselines, so that the value of controlling computation can be compared with the value of the planning model.

III. PROBLEM FORMULATION

A central planner is called whenever a replanning trigger fires. Each UAV keeps the task that it is currently flying to or serving, which determines when and where it becomes available again and how much energy it has left. Each open task j has a location, a release time r_j, a deadline d_j by which its service must be completed, a service duration and a reward w_j. A plan assigns an ordered sequence of open tasks to each UAV, and tasks may remain unassigned, so the planning problem is a prize-collecting routing problem with time windows [20], [21]. Arrival, start and finish times along a route are computed as usual for time windows [44]. A route is feasible if every service ends before its deadline and the UAV returns to the depot with an energy reserve of 10% of its capacity. A plan π is evaluated by

$$J(\pi) = \alpha R(\pi) - \gamma E(\pi) - \delta C(\pi), \tag{1}$$

where R is the reward of the tasks served on time, E the flown distance in kilometres and C the number of tasks that move to another UAV than in the previous plan, with $\alpha = 1$, $\gamma = 0.05$ per kilometre and $\delta = 0.3$. W denotes the total reward of the open tasks.

Computation is not free. A plan computed for τ milliseconds is released $\kappa\tau$ milliseconds later in mission time, where the latency factor κ represents planning computers slower than the evaluation computer. An idle UAV waits for the new plan, so its route starts later, and tasks whose deadline can no longer be met are lost. The plan then reaches the vehicles after a dissemination latency L that the planner does not control.

The planner uses a nominal model with straight legs at the cruise speed of each UAV and nominal service durations. The executed finish time of task j is $F_j = f_j + e_j$, where f_j is the nominal finish time at publication and e_j an execution error caused by acceleration, wind, speed tracking and service-time variability. A task yields its reward only if $F_j \leq d_j$.

TABLE 2. Notation.

Symbol	Meaning
t_0	time of the planning snapshot
r_j, d_j, w_j	release time, deadline and reward of task j
f_j, F_j	planned (nominal) and actual finish time of task j
$h_j = f_j - t_0$	lead time of task j
$e_j = F_j - f_j$	finish-time error, $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$
$m(h)$, $s(h)$	mean and spread of the error, $b_0 + b_1 h$ and $s_0 + s_1 h$
z	safety factor; protected tasks meet d_j with probability $\geq \Phi(z)$
$\tilde{d}_j(z)$	effective deadline, Eq. (4)
$\bar{H}$, H	tasks no UAV can finish by $\tilde{d}_j$, resp. by d_j, on a direct flight
λ	cost of a late arrival, in units of the task reward
$\theta(\lambda)$, z_λ	break-even probability $\lambda/(1+\lambda)$ and $\Phi^{-1}(\theta(\lambda))$
d'_j	planning deadline used by repair and search
τ, B	planning time of a call and its hard budget
κ, L	latency factor and communication latency
u_λ	score $U - \lambda n_{\text{late}}/n$ of a mission

As in dispatching practice, a UAV skips a task at departure if the nominal estimate from its actual state already misses the deadline, and the task returns to the pool. The utility of a mission is the reward of all tasks served on time divided by the total reward of the mission. Three replanning triggers are considered. The event trigger replans on task arrivals, UAV failures and deadline changes, the periodic trigger in addition every T seconds, and the deviation trigger in addition when a finish time deviates from the plan by more than θ seconds. During planning, every UAV finishes its committed task and then waits, and events are queued.

IV. PROPOSED METHOD

LTCC (lead-time-scaled chance constraints) takes four decisions in every replanning cycle (Fig. 1, Algorithm 1). The chance-constrained planning model decides which plans are acceptable under the calibrated execution error. The cost-aware fallback decides which of the tasks that this model gives up are still worth attempting, given the cost of a late arrival. The stopping rule decides when to stop computing and publish, and the trigger decides when to replan. The first two decisions are the contribution of this paper. Both reduce to one effective deadline per task, so a feasibility-first repair and an anytime ALNS, or any other heuristic that checks deadlines, produce the plan unchanged. Table 2 lists the notation. Two earlier variants serve as baselines. AnytimeSwarm uses the same stopping rule with a deterministic delay cost on the nominal model, and AnytimeSwarm-R uses an execution-aware delay cost on the nominal model.

A. REPAIR AND ANYTIME SEARCH

The previous plan is restricted to the open tasks and walked once, keeping a task only if the route up to it is still feasible, and tasks of failed UAVs and new tasks are inserted by parallel regret-2 insertion [5], [45]. This feasibility-first repair yields a publishable plan within milliseconds. ALNS [5] then improves it with four destroy operators (random, worst, related [46] and route-segment removal), four greedy or regret-2 repair operators with and without noise, adaptive operator selection and simulated-annealing acceptance [47]. All intermediate solutions are feasible, so the best plan found so far can be published at any time. The temperature decreases per iteration and all variants share the random seed, so every stopping rule follows the same search trajectory and differs only in the moment at which it stops.

Algorithm 1 One replanning call of LTCC

Require: snapshot at time t_0: committed tasks, open tasks with (r_j, d_j, w_j), UAV states; previous plan π^-; error model (b_0, b_1, s_0, s_1), safety factor z, late cost λ, expected latency $\bar{L}$, latency factor κ

$z_\lambda \leftarrow \Phi^{-1}(\lambda/(1+\lambda))$; $z_p \leftarrow \max\{z, z_\lambda\}$ ▷ $z_\lambda = -\infty$ for $\lambda = 0$

$\tilde{d}_j(\cdot) \leftarrow \min\{d_j,\ t_0 + (d_j - t_0 - b_0 - \cdot\, s_0)/(1 + b_1 + \cdot\, s_1)\}$ for every open task ▷ Eq. (4)

$d'_j \leftarrow \min\{d_j, \tilde{d}_j(z_\lambda)\}$ if no UAV can finish j by $\tilde{d}_j(z_p)$ but one can by d_j, else $d'_j \leftarrow \tilde{d}_j(z_p)$ ▷ Eq. (7)

$\pi \leftarrow$ FeasibilityFirstRepair(π^-, d'); $\pi^* \leftarrow \pi$; $\tau \leftarrow 0$

repeat

  destroy and repair π with adaptively chosen operators; accept by simulated annealing

  if $J(\pi) > J(\pi^*)$ **then** $\pi^* \leftarrow \pi$

  every 5 ms: $\widehat{\Delta J} \leftarrow$ learned prediction of the gain in the next H ms

until $\widehat{\Delta J} \leq C_{\mathrm{R}}(\tau) + \mu W H$ **or** $\tau \geq B$ ▷ Eq. (8)

publish π^*; it reaches the UAVs at $t_0 + \kappa\tau + L$

next call at the next event or $T_{\max}$ after t_0, whichever comes first

B. CALIBRATED FINISH-TIME ERROR

The error of task j is modelled as $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$, where $h_j = f_j - t_0$ is its lead time, the nominal finish time minus the time t_0 of the planning snapshot, and

$$m(h) = b_0 + b_1 h, \qquad s(h) = s_0 + s_1 h, \tag{2}$$

because errors accumulate along a route. The parameters are fitted per disturbance level on development missions, the mean by least squares and the spread by least squares on the absolute residuals scaled by $\sqrt{\pi/2}$, which is less sensitive to the heavy tail of the errors. Under the mid and high disturbances of the main studies, both the mean and the spread grow with the lead time (Supplementary Table S2). The nominal model is therefore optimistic, and increasingly so along a route.

The affine spread is a modelling choice that the data can reject. A route-level model with independent increments per unit of time, the usual assumption when variances are added along a route, implies $s(h) = \sqrt{q_0 + q_1 h}$. On 4 982 and 4 231 pairs of lead time and error from 90 development missions per level, the affine spread fits better by 859 and 910 points of the Akaike information criterion under mid and high disturbances, and also better than the hybrid $\sqrt{q_0 + q_1 h + q_2 h^2}$ (by 72 and 117). A five-fold cross-validation with missions as groups gives the same ranking. At the longest lead times, about 514 s, the square-root model predicts a spread of 262 s under high disturbances where 372 s are observed, while the affine model predicts 380 s (Fig. 2a).

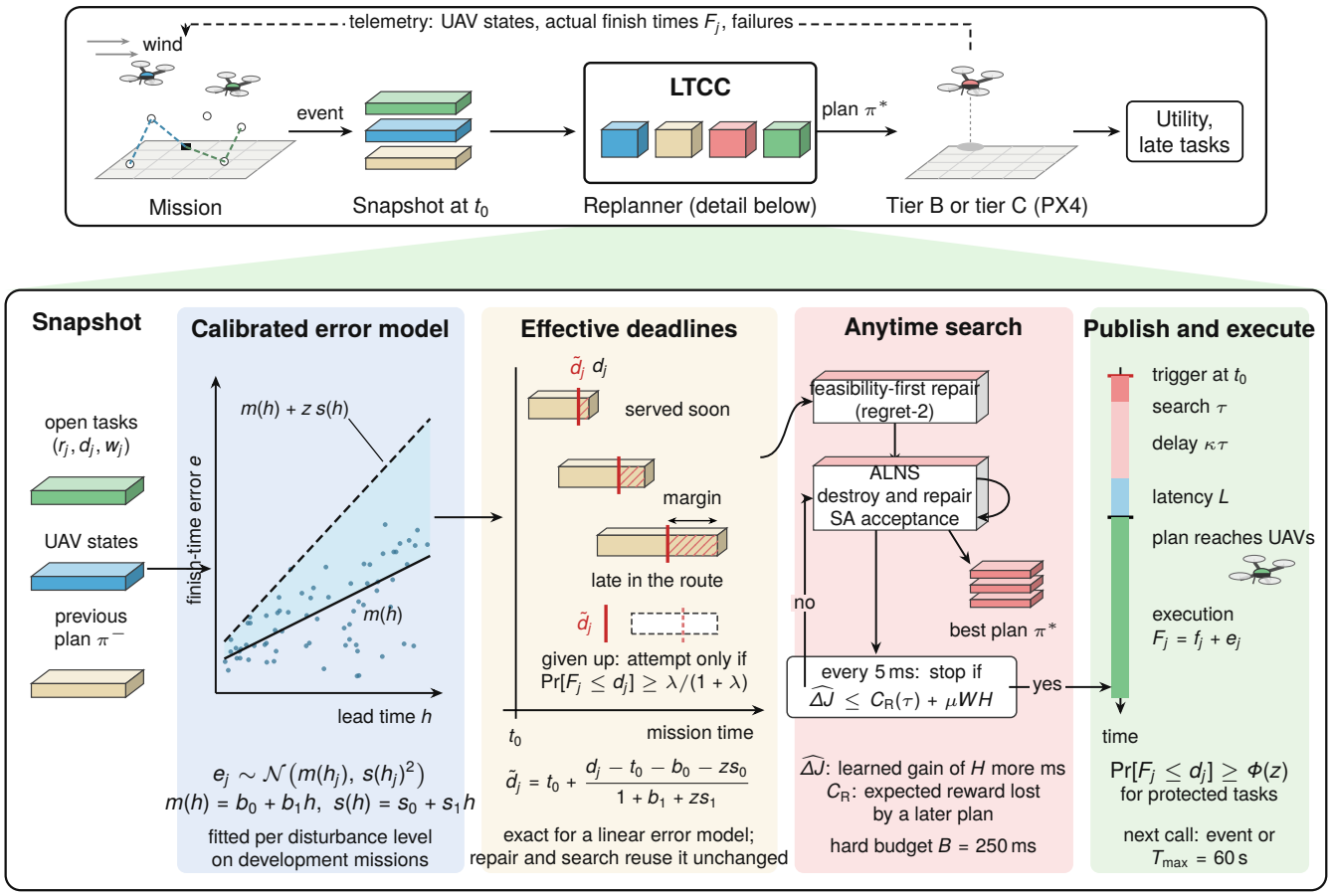


FIGURE 1. **Overview of LTCC. Top: the replanning loop between the mission, the planning snapshot, the replanner and the execution tier. Bottom: one replanning call. The calibrated error model gives a margin that grows with the lead time, the chance constraint turns it into an effective deadline per task, the cost-aware fallback keeps a given-up task only if its probability of meeting the deadline outweighs the cost λ of a late arrival, and the anytime search stops when the predicted gain no longer pays for the delay. The plan is released after the emulated planning delay $\kappa\tau$ and the latency L.**

The reason is that the disturbances persist over a leg. Wind and an airspeed error act during the whole flight to a task, so the error of a leg is proportional to its duration (Fig. 2b), with a coefficient of variation of 0.39 and 1.15 under mid and high disturbances. Errors of successive legs are close to independent. The correlation between the error of a task and the error added by the next task of the same plan was -0.20 and -0.05, and missions explained only 5% and 7% of the residual variance. A lead time of a few legs therefore carries a spread that grows almost linearly. Moment propagation with a variance proportional to the squared duration of each leg is consistent with this structure and serves as the literature baseline in Section V-C.

C. CHANCE-CONSTRAINED PLANNING MODEL

The proposed model plans for the calibrated execution. A planned finish time f_j is accepted only if

$$f_j + m(f_j - t_0) + z\,s(f_j - t_0) \;\le\; d_j, \tag{3}$$

where $z \ge 0$ is a safety factor. The margin $m + zs$ depends on the finish time itself, so it is small for tasks served soon and large for tasks served late in a route. Because m and s are linear, the constraint reduces to an effective deadline,

$$f_j \le \tilde{d}_j, \qquad \tilde{d}_j = t_0 + \frac{d_j - t_0 - b_0 - z s_0}{1 + b_1 + z s_1}. \tag{4}$$

The repair and the search use $\tilde{d}_j$ instead of d_j, while the executor and the reward still use the true deadline.

Proposition 1: Let $1 + b_1 + z s_1 > 0$. (i) A finish time satisfies Eq. (3) if and only if $f_j \le \tilde{d}_j$. (ii) Under the error model of Eq. (2), a task planned to finish at $f_j \le \tilde{d}_j$ meets its deadline with probability at least $\Phi(z)$ if its route starts as planned. (iii) With $b_0 = b_1 = s_0 = s_1 = 0$, $\tilde{d}_j = d_j$ and the planning model is the nominal one.

Proof: Substituting Eq. (2) into Eq. (3) gives $f_j(1 + b_1 + z s_1) \le d_j - b_0 - z s_0 + (b_1 + z s_1) t_0$, and dividing by the positive factor gives (i). For (ii), the actual finish is $F_j = f_j + e_j$ with $e_j \sim \mathcal{N}(m(h_j), s(h_j)^2)$ and $h_j = f_j - t_0$. By (i), $d_j - f_j - m(h_j) \ge z\,s(h_j)$, hence $\Pr[F_j \le d_j] = \Phi\big((d_j - f_j - m(h_j))/s(h_j)\big) \ge \Phi(z)$. Part (iii) is immediate. ■

The effective deadline turns a chance constraint on every task into a deterministic constraint of the same form as the original one. The insertion heuristics and the feasibility

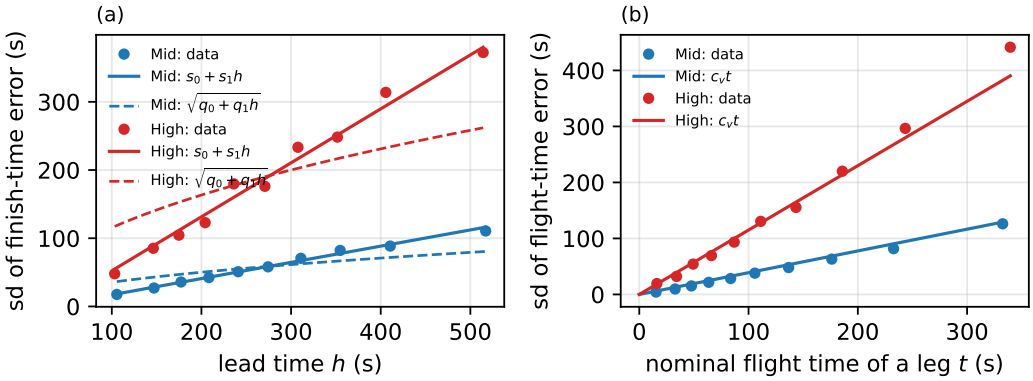


FIGURE 2. Structure of the finish-time error on development missions. (a) Standard deviation of the error against the lead time, in deciles, with the affine fit of Eq. (2) and the fit with independent increments. (b) Standard deviation of the flight-time error of single legs against their nominal flight time, in deciles, with a spread proportional to the duration.

checks of the search are therefore reused unchanged, and computing $\tilde{d}_j$ costs one division per task. With $z = 0$, the model corrects the mean error only and plans for the median finish time. Larger values of z protect more tasks against lateness but leave fewer tasks feasible, so z trades the risk of a late service against the reward of attempting a task. It is chosen per disturbance level on development missions.

D. NOMINAL FALLBACK FOR GIVEN-UP TASKS

A hard chance constraint has a side effect when the time windows are short compared with the spread of the error. If no vehicle can finish task j before $\tilde{d}_j$, even on a direct flight, the task is infeasible for the planner and is never attempted. A vehicle may still serve it in time with a probability below $\Phi(z)$ but well above zero, and it may have nothing better to do. The fallback keeps such tasks. Let $\tilde{H}$ be the set of tasks that no vehicle can finish by $\tilde{d}_j$ on a direct flight, and H the set of tasks that no vehicle can finish by d_j. The planning deadline is

$$d'_j = \begin{cases} d_j, & j \in \tilde{H} \setminus H, \\ \tilde{d}_j, & \text{otherwise.} \end{cases} \tag{5}$$

The constraints of all other tasks are unchanged, so every other task that is served keeps the guarantee of Proposition 1, while a fallback task carries none. Finding $\tilde{H}$ and H requires one direct-flight check per task and vehicle, which is negligible compared with the search. The fallback was designed after the held-out results of study 2 and is evaluated on new seeds in studies 3 and 4.

E. COST-AWARE FALLBACK

The nominal fallback is right only if a late service costs nothing. In many applications a late arrival has a cost of its own. The flight time and energy are spent, a customer may have left, or a commitment is broken. Let a late arrival at task j cost λw_j, so that λ measures this cost in units of the reward of the task. An attempt that meets the deadline with probability p then has the expected value $p\,w_j - (1-p)\lambda w_j$, which is non-negative if and only if

$$p \geq \theta(\lambda) = \frac{\lambda}{1+\lambda}. \tag{6}$$

By Proposition 1, a finish time $f_j \leq \tilde{d}_j(z)$ gives a probability of at least $\Phi(z)$, where $\tilde{d}_j(z)$ denotes the effective deadline of Eq. (4) at safety factor z. The break-even probability therefore corresponds to the safety factor $z_\lambda = \Phi^{-1}(\theta(\lambda))$. The cost-aware fallback protects every task at $z_p = \max(z, z_\lambda)$, and plans a task that no vehicle can finish by $\tilde{d}_j(z_p)$ on a direct flight against $\min\{d_j, \tilde{d}_j(z_\lambda)\}$. A task that cannot meet this deadline either is given up. Thus

$$d'_j = \begin{cases} \min\{d_j, \tilde{d}_j(z_\lambda)\}, & j \in \tilde{H}(z_p) \setminus H, \\ \tilde{d}_j(z_p), & \text{otherwise,} \end{cases} \tag{7}$$

with $\tilde{H}(z_p)$ the set of tasks that no vehicle can finish by $\tilde{d}_j(z_p)$ on a direct flight.

Proposition 2: Let $s_0, s_1 \geq 0$, let $1 + b_1 + z s_1 > 0$ for every finite z used, and read $\tilde{d}_j(-\infty)$ as d_j. (i) For $\lambda = 0$, Eq. (7) is the nominal fallback of Eq. (5). (ii) For $\lambda \geq \Phi(z)/(1-\Phi(z))$, no task falls back, and every planned task meets $\tilde{d}_j(z_\lambda)$. (iii) Under the error model of Eq. (2), every task planned to finish by d'_j has an expected contribution $w_j \Pr[F_j \leq d_j] - \lambda w_j \Pr[F_j > d_j] \geq 0$ if its route starts as planned. (iv) Every planning deadline d'_j is non-increasing in λ.

Proof: (i) $z_\lambda = -\infty$ gives $z_p = z$ and $\min\{d_j, \tilde{d}_j(-\infty)\} = d_j$. (ii) $z_\lambda \geq z$ holds if and only if $\theta(\lambda) \geq \Phi(z)$, that is, $\lambda \geq \Phi(z)/(1-\Phi(z))$. Then $z_p = z_\lambda$, and a task that cannot meet $\tilde{d}_j(z_p)$ cannot meet $\min\{d_j, \tilde{d}_j(z_\lambda)\}$ either, so it is given up. (iii) A protected task meets its deadline with probability at least $\Phi(z_p) \geq \Phi(z_\lambda)$ and a fallback task with probability at least $\Phi(z_\lambda)$, both by Proposition 1. Hence $p \geq \theta(\lambda)$, and the expected contribution $w_j\big((1+\lambda)p - \lambda\big)$ is not negative. (iv) With $s_0, s_1 \geq 0$, $\tilde{d}_j(z)$ does not increase with z, and z_λ and z_p do not decrease with λ. A fallback task stays one when λ grows, because z_p does not decrease. A protected task can

become a fallback task only while $z_\lambda < z$, and then $z_p = z$ does not change. In each case the deadline does not increase. ■

For λ between these limits, the rule attempts a given-up task only if its expected contribution is not negative. It adds no parameter to tune, because z keeps its value and λ is a property of the application. Where $1 + b_1 + z_\lambda s_1 \leq 0.05$, which occurs only for $\lambda < 0.03$, the nominal deadline is used. The rule needs only a probability of meeting the deadline, so it applies to any planning model that provides one, such as moment propagation, where the given-up tasks are accepted if $\mathrm{E}[F_j] + z_\lambda \mathrm{sd}[F_j] \leq d_j$. It ignores the delay that a fallback task causes to the tasks after it on the same route. A margin without a probability of meeting the deadline, such as a proportional buffer, has no counterpart of z_λ, so its fallback deadline has to be tuned for every value of λ. The cost-aware fallback was designed after the held-out results of study 4 and is evaluated on new seeds in studies 5 and 6.

F. STOPPING RULE AND TRIGGER

Every 5 ms, the stopping rule compares the predicted gain $\widehat{\Delta J}$ of searching $H = 10$ ms longer with the cost of doing so, and stops at time τ if

$$\widehat{\Delta J} \;\leq\; C_{\mathrm{R}}(\tau) + \mu W H, \tag{8}$$

with a price of computation $\mu = 10^{-5}$ per millisecond. The gain is predicted from eleven features of the search history by a Poisson generalised linear model [48] fitted on development trajectories. The delay cost $C_{\mathrm{R}}(\tau) = \Lambda_{\mathrm{R}}(\tau + H) - \Lambda_{\mathrm{R}}(\tau)$ is the increase of the expected value of the tasks that a later start would lose,

$$\Lambda_{\mathrm{R}}(\tau) = \sum_j v_j \, \Phi\left(\frac{\sigma_j(\tau) + \bar{L} + m(h_j) - \mathrm{tol}_j}{s(h_j)} \right), \tag{9}$$

where $\sigma_j(\tau)$ is the start shift of the route of task j caused by waiting for the plan, tol_j the margin of the task to its deadline plus the waiting times before it on its route [44], $\bar{L}$ the expected latency and v_j the value of the task in Eq. (1). On the effective deadlines, the mean error is already part of the tolerance, so LTCC uses $m \equiv 0$. Λ_{R} does not decrease with τ, and it reduces to the exact loss of a deterministic plan when the spread vanishes (Supplementary Section S1). A hard budget B bounds every call. The replanner replans on every event and at least every $T_{\max} = 60$ s of mission time, a rolling-horizon policy in the terminology of rescheduling [29], [30].

V. EVALUATION

A. SCENARIOS AND KINEMATIC TIER

Missions are created by a seeded generator (Supplementary Table S1). The map is a square with a side of 10 km and the depot at its centre, half of the tasks are clustered around three hotspots, and half of them are known at the start while the others arrive in batches. Time windows have a slack of 180 to 600 s and services last 30 to 90 s. Three fleet scales with 5, 10 and 20 UAVs and 25, 50 and 140 tasks are combined with three event profiles, namely six arrival batches, three batches with two UAV failures, and three batches with four tightened deadlines.

Tier B executes each published plan in continuous mission time. Legs follow a trapezoidal speed profile with an acceleration limit, the airspeed of each leg deviates from the nominal speed by a multiplicative error, a constant wind of random direction changes the ground speed, and service durations are multiplied by a random factor (Table 3). The planner never sees these disturbances and receives the state of each UAV as telemetry. The dissemination latency is lognormal, and lost messages are resent after a timeout. All random draws use common random numbers [49] keyed to the mission seed, the UAV and the task, so two planners that assign the same task to the same UAV experience the same disturbance.

B. SOFTWARE-IN-THE-LOOP TIER

Tier C flies missions with three X500 quadrotors, each controlled by its own PX4 v1.17 autopilot in software-in-the-loop mode [17], in Gazebo Harmonic [18] on ROS 2 Jazzy [19] (Fig. 3). The mission manager runs the scenario timeline, builds the planning snapshot, calls the planner and releases the plan. The executor streams offboard setpoints to every autopilot at 10 Hz through a uXRCE-DDS bridge, flies each leg with a carrot that moves along the straight leg and holds the UAV at the task for the service duration. A tier C mission is a tier B scenario with three UAVs and fifteen tasks, rescaled by $S_X = 0.1$ in length and $S_T = 0.25$ in time. Speeds are multiplied by S_X/S_T, energy by S_X and the distance price by $1/S_X$, so every dimensionless ratio of the decision problem is preserved, and $\kappa_C = S_T \kappa_B$. The commanded ground speed of each leg follows the speed error and the scaled wind of tier B with the same random numbers, while acceleration, attitude dynamics, position tracking and communication are those of the simulated vehicles. Gazebo runs in lockstep with PX4 at twice real time, and mission time is simulation time.

C. COMPARED METHODS

All methods share the feasibility-first repair, the ALNS operators and parameters, the initial plan and the random seed, and every call has a hard budget of 250 ms. They differ in the stopping rule, the planning model and the trigger. Regret-2 repair without search is the constructive floor. ALNS is run with fixed budgets of 250 ms and 40 ms [5], the latter equal to the mean computation of AnytimeSwarm on development data, with a budget of 1 ms per open task, and with a stagnation rule that stops after 20% of the budget without improvement. Two stopping rules from the metareasoning literature are re-implemented with the same delay cost and price of computation as the proposed rule, so that only the prediction of the gain differs. The performance profile rule [13], [16] predicts the gain from a static profile over elapsed time and number of open tasks, and the online prediction rule [14] fits $q(t) = a - b\,e^{-ct}$ to the best objective found so far at every check. The constant-buffer model subtracts 60 s from every deadline, chosen on development missions among 30, 60 and

TABLE 3. Disturbance and latency levels of tier B. The levels span mild to severe conditions and are not claimed to represent a particular vehicle.

Execution	accel. (m/s^2)	speed c.v.	wind (m/s)	service c.v.	Latency	median (s)	$\sigma_{\log}$	loss / timeout
low	3.0	0.03	2	0.05	low	0.2	0.5	–
mid	2.0	0.07	5	0.15	high	2.0	0.8	5% / 5 s
high	1.5	0.12	8	0.25				

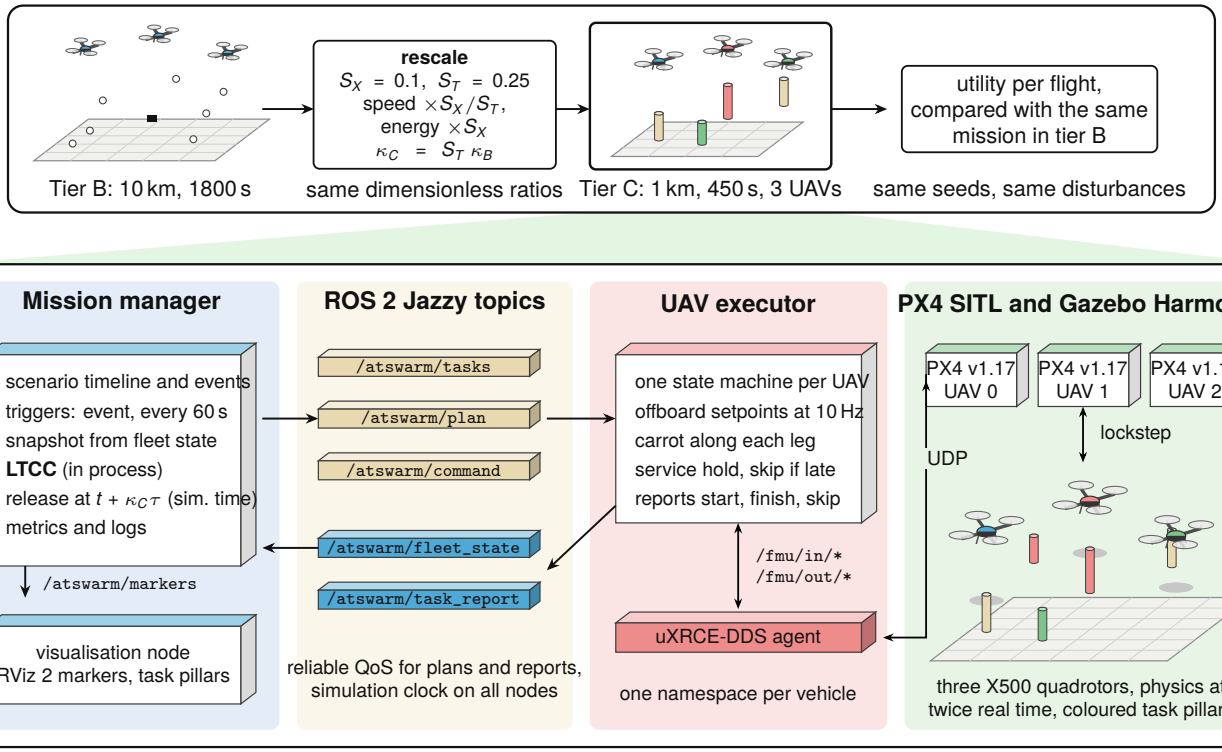


FIGURE 3. Software-in-the-loop tier. Top: a tier B mission is rescaled in length and time so that the decision problem is unchanged. Bottom: the planner runs inside the ROS 2 mission manager, the executor streams offboard setpoints through the uXRCE-DDS bridge to three PX4 autopilots, and Gazebo simulates three X500 quadrotors in lockstep.

120 s, as is common for robust schedules with time buffers [7]. The proposed model uses Eq. (4) with $z = 0.5$ under mid and $z = 0$ under high disturbances, selected on development missions. All these methods use the periodic trigger. The conventional planner combines ALNS with 40 ms, the nominal model and the event trigger.

Study 4 compares planning models on the same search, with a fixed budget of 40 ms, the periodic trigger and the nominal fallback for every chance-constrained model. The proportional buffer accepts $f_j + c\,h_j \le d_j$, which keeps the shape of the proposed margin but replaces the calibration by one tuned slope. Moment propagation follows Ehmke et al. [8]. The flight time of a leg of nominal duration t is normal with mean $(1+a)t+c_0$ and standard deviation $c_v t$, the service time is normal with mean $(1 + a_s)s$ and standard deviation $c_s s$, legs are independent, the start of service $\max(A_j, r_j)$ is approximated by Clark's moments [38], and a finish is accepted if $\mathrm{E}[F_j]+z\,\mathrm{sd}[F_j] \le d_j$. The leg parameters are fitted on the legs of the development flights. Because the variance of one long leg can exceed that of the two legs it replaces, removing a task can make a route infeasible under this model, so every route is filtered once more before publication. Online calibration starts from one error model fitted on both disturbance levels, with the weight of n_0 pairs, and refits Eq. (2) at every call with the pairs of the tasks completed so far in the mission, so it needs neither the disturbance level nor a level-specific z. Each model has one parameter per level, selected on development missions by the same rule as z in study 2. For the proposed model with fallback, z was selected anew and is 1 under mid and 0.5 under high disturbances. The other values are $c = 0.45$ and 0.6, $z = 1$ and 0.5 for moment propagation, and $n_0 = 50$ with $z = 0.5$ for online calibration. On development missions, the three static models differed by less than 0.2 pp.

Study 5 charges every late arrival with λ times the mean task reward of the mission and scores a run by $u_\lambda = U - \lambda\, n_{\mathrm{late}}/n$, where U is the utility, n_{late} the number of tasks served late and n the number of tasks. $\lambda = 0$ gives the utility of studies 1–4. The cost-aware fallback of Section IV-E is run with the λ it is scored with and the z of study 4,

without any further tuning. Its competitors are tuned for every $\lambda \in \{0.5, 1, 2\}$ and disturbance level on development missions. These are the proportional buffer with $c \in \{0.1, 0.2, 0.3, 0.45, 0.6, 0.75, 0.9, 1.2\}$, with or without the nominal fallback, and the proposed model without fallback with $z \in \{-0.5, 0, 0.5, 1, 1.5\}$. A grid that first ended at $c = 0.3$ and $z = 0$ was extended after both values were selected on its edge, and the selection did not change. For $\lambda = 1$, the selected buffer was $c = 0.3$, no fallback under mid and $c = 0.45$, no fallback under high disturbances, and the selected model without fallback had $z = 1$ and $z = 0.5$.

Study 6 adds a published task-allocation algorithm and applies the cost-aware fallback to moment propagation. The consensus-based bundle algorithm (CBBA) [4] is re-implemented with bids equal to the largest increase of the route value over all feasible insertion positions, a fully connected network, and the bid warping of Johnson et al. [50]. Time windows break the diminishing marginal gains on which the convergence of CBBA rests, and without bid warping it did not converge on some development snapshots. CBBA plans from scratch at every call, stops at convergence or at the hard budget, and pays for its planning time like every other method. It runs with the periodic and with the event trigger, and also on the deadlines of the cost-aware fallback with $\lambda = 1$. Moment propagation with the cost-aware fallback accepts a given-up task if its nominal finish meets the deadline and $\mathrm{E}[F_j] + z_\lambda \mathrm{sd}[F_j] \le d_j$. No parameter was tuned in study 6.

D. PRE-REGISTERED PROTOCOL

The evaluation consists of six studies, each with its own held-out seeds, written protocol and frozen code, and each study after the first was designed after the held-out results of the earlier ones. Table 4 lists their primary hypotheses. Development used seeds 0–9 in tier B and 0–2 in tier C, on which the predictor, the compute price, z, the buffers and the error models were fixed. Before each study, the configurations, the models and the code were frozen with SHA-256 hashes, which the held-out runs checked before starting.

Study 1 tested AnytimeSwarm-R against fixed and adaptive stopping rules on the nominal model (Supplementary Section S4). Studies 2–6 share one tier B matrix. It crosses two disturbance levels (mid, high), high latency, three latency factors ($\kappa_B = 10, 50, 200$) and two time-window factors (original and halved), which gives twelve configurations of 270 held-out missions each, with new seeds 5000–5029, 7000–7029, 8000–8029, 10000–10029 and 12000–12029 for studies 2, 3, 4, 5 and 6. Study 2 compares fifteen methods (Supplementary Table S6), and its tier C part flies 30 missions with mid disturbances and $\kappa_B = 50$ (seeds 6000–6009). Study 4C flies 36 missions under high disturbances with halved windows, where tier B predicted the largest effect (seeds 9000–9011). Its size was computed beforehand from a tier B counterpart on development seeds, and one interim analysis after half of the missions uses O'Brien–Fleming bounds, so that the flights could stop early without inflating the error rate. Studies 5 and 6 test their primary hypotheses with the score u_1.

In all studies, the utility difference of two methods is averaged per mission over the configurations and tested with a paired bootstrap [51] of 10 000 resamples over missions. Primary families are corrected with the Holm procedure [52]. Non-inferiority uses a margin of -0.5 pp and is followed by a superiority test in a fixed sequence, and equivalence is tested with two one-sided tests. All other comparisons are descriptive. A tier C run in which a vehicle does not take off in time, or whose real-time factor leaves $[1.6, 2.2]$, is repeated once. Tier B runs four missions in parallel on an Intel Core i7-11800H laptop, with the methods of a configuration run in turn. A separately written validator found no infeasible plan among 4 779 development plans. The protocols and their deviations are available with the code.

VI. RESULTS

Table 4 summarises the pre-registered primary hypotheses of studies 2–6 and their outcomes. The subsections report each study in the order in which it was designed, so that every design decision can be traced to the results that motivated it.

A. STUDY 1 IN BRIEF: EXECUTION-AWARE STOPPING ON THE NOMINAL MODEL

Study 1 made only the stopping rule execution-aware and kept the nominal planning model. In 34 560 tier B runs, AnytimeSwarm-R needed 15% less computation per call than AnytimeSwarm, but its advantage over ALNS with 40 ms was +0.32 [+0.01, +0.63] percentage points (pp) with a Holm-adjusted p of 0.128. Periodic replanning raised the utility of every method by +1.29 to +1.56 pp. Supplementary Sections S3 and S4 report these results in full, together with an exact-execution benchmark, tier C flights and a scalability study with 40 UAVs. Study 2 moved the error model from the decision to stop into the planning model.

B. STUDY 2: THE CHANCE-CONSTRAINED MODEL AGAINST BASELINES

Study 2 comprises 48 600 tier B runs, that is, 270 held-out missions in 12 configurations with 15 methods (Supplementary Table S6). The proposed replanner, still without fallback, reached the highest utility, 52.15%. Its advantage over the six baselines of H1 ranged from +1.35 pp over the constant buffer to +2.87 pp over ALNS with 40 ms (all Holm-adjusted $p < 0.001$), and over the conventional planner it was +4.44 [+3.82, +5.05] pp, or 9.3% (H2). The proposed replanner served 5.72 tasks per mission after their deadline, 49% fewer than the 11.22 of ALNS with 40 ms. On the nominal model, the stopping rules hardly differed, with utilities between 49.28% and 49.66% although their computation ranged from 11.2 to 58.1 ms per call (Supplementary Fig. S4). On the chance-constrained model, the stopping rule was equivalent to a fixed budget of 40 ms (H3, 90% interval +0.22 [+0.09, +0.35] pp inside ± 0.5 pp), but it still guarded against long

TABLE 4. Pre-registered primary hypotheses of studies 2–6 on held-out missions and their outcomes. Differences are the first method minus the second, in percentage points with 95% paired bootstrap intervals; U is the utility and u_1 the score with a late cost of one task reward. Primary families are Holm-corrected; NI: non-inferiority with a margin of -0.5 pp; sup.: superiority tested after NI in a fixed sequence. Study 1 is reported in Supplementary Section S4.

Study	H	Comparison	Score	Difference (pp)	Outcome
2	H1	proposed, no fallback, vs. six baselines	U	+1.35 to +2.87	supported, 6 of 6
	H2	proposed, no fallback, vs. conventional	U	+4.44 [+3.82, +5.05]	supported
	H3	stopping rule vs. fixed 40 ms (equivalence)	U	+0.22 [+0.07, +0.37]	equivalent
	H4	tier C: proposed vs. ALNS 40 ms	U	−0.25 [−4.02, +3.57]	not supported
3	H5	nominal fallback vs. no fallback	U	+1.97 [+1.61, +2.33]	supported
	H6	nominal fallback vs. buffer, high, windows $\times 0.5$	U	+3.31 [+2.58, +4.03]	supported
	H7	nominal fallback vs. ALNS 40 ms	U	+4.61 [+4.15, +5.06]	supported
	H8	nominal fallback vs. constant buffer	U	+3.59 [+3.17, +4.02]	supported
4	H9	proposed vs. proportional buffer	U	−0.38 [−0.72, −0.05]	not supported
	H10	proposed vs. moment propagation (NI)	U	−0.25 [−0.62, +0.12]	NI not shown
	H11	nominal fallback within moment propagation	U	+9.66 [+9.04, +10.29]	supported
	H12	proposed vs. constant buffer	U	+4.22 [+3.74, +4.68]	supported
	H13	proposed vs. nominal model	U	+5.07 [+4.63, +5.52]	supported
4C	H14	tier C: proposed vs. nominal model	U	−0.57 [−3.49, +2.29]	not supported
	H15	tier C: nominal fallback vs. no fallback	U	+10.35 [+6.41, +14.39]	supported
5	H16	cost-aware vs. nominal fallback	u_1	+2.42 [+1.81, +3.02]	supported
	H17	cost-aware vs. proportional buffer tuned for λ	u_1	+0.90 [+0.38, +1.44]	supported, sup.
	H18	cost-aware vs. no fallback, z tuned for λ	u_1	+1.78 [+1.25, +2.31]	supported, sup.
	H19	cost-aware vs. nominal model	u_1	+11.75 [+10.88, +12.57]	supported
6	H20	proposed vs. CBBA	u_1	+12.77 [+11.91, +13.63]	supported
	H21	CBBA on the proposed model vs. CBBA	u_1	+12.12 [+11.31, +12.93]	supported
	H22	moment prop.: cost-aware vs. nominal fallback	u_1	+2.71 [+2.13, +3.27]	supported
	H23	moment prop.: cost-aware vs. no fallback	u_1	+3.30 [+2.83, +3.79]	supported

budgets: at $\kappa_B = 200$, the full budget of 250 ms lost 23.42 tasks per mission to late plans.

The gain was not uniform. The proposed replanner was worse than the constant buffer in 3 of the 12 configurations, all of them under high disturbances with halved windows, where it served −2.01 [−2.80, −1.21] pp relative to the buffer. With short windows and large errors, the effective deadline of many tasks falls before the earliest possible arrival, so the planner gives up tasks that a less cautious plan still serves in time. This weakness motivated the fallback of Section IV-D.

C. STUDY 3: NOMINAL FALLBACK FOR GIVEN-UP TASKS

Study 3 comprises 16 200 tier B runs on 270 new held-out missions (Supplementary Table S9). The replanner of study 2 again lost to the constant buffer under high disturbances with halved windows, by −1.95 [−2.72, −1.20] pp, so the weakness is systematic. The fallback variant reached the highest utility, 54.40%, and all four primary comparisons were significant (Holm-adjusted $p < 0.001$). It served +1.97 [+1.61, +2.33] pp more than the replanner of study 2 (H5), +3.31 [+2.58, +4.03] pp more than the constant buffer in the weak regime (H6), and +4.61 [+4.15, +5.06] and +3.59 [+3.17, +4.02] pp more than ALNS with 40 ms and the constant buffer over all configurations (H7, H8). The fallback has a price in punctuality. A fallback task is planned against its nominal deadline and is often served late, and the fallback variant served 8.33 tasks late per mission, 46% more than the replanner of study 2. The case in which a late arrival has a cost of its own is the subject of study 5.

D. STUDY 4: PLANNING MODELS AND THE GENERALITY OF THE FALLBACK

Study 4 comprises 35 640 tier B runs on 270 new held-out missions (Table 5). All planning models use the same search with 40 ms and the periodic trigger, so they differ only in the margin and the fallback. The study replicates the main result on a third set of held-out seeds. With the fallback, the proposed model served 4.22 pp more reward than the constant buffer (95% CI 3.74 to 4.68, H12) and 5.07 pp more than the nominal model (4.63 to 5.52, H13), that is, 8.5% and 10.4% more, and 13.6% more than the conventional planner. All three advantages held in each of the four regimes (Supplementary Fig. S8), and with repair only and no ALNS the proposed model still served 3.33 pp more than the nominal one. Across the three independent sets of held-out seeds, every advantage over the constant buffer, the nominal model and the conventional planner was significant, and each grew once the fallback was added (Fig. 4).

The three models whose margin grows with the lead time reached utilities within 0.38 pp of each other. The proportional buffer served 0.38 pp more than the proposed model (−0.72 to −0.05, Holm-adjusted $p = 0.050$), so H9 is not supported. Moment propagation served 0.25 pp more, and the interval of the proposed model minus moment propagation, −0.62 to 0.12, crossed the margin of −0.5 pp, so non-inferiority was not shown (H10). The intercept and the mean correction of the calibrated model therefore add no utility beyond a tuned proportional margin. The closed form was cheaper, with 1.13 more search iterations per millisecond

TABLE 5. Study 4, tier B held-out results: utility (%) per planning model overall and per regime (disturbance level and time-window factor, mean over κ_B), tasks served late per mission and search iterations per planning millisecond. All models use the same search with a fixed budget of 40 ms and periodic replanning. 270 missions, 12 configurations.

		Mid		High			
Planning model	All	×1	×0.5	×1	×0.5	Late	it/ms
Proposed planning model	53.85	73.07	48.05	57.01	37.27	7.54	13.35
Proposed, online calibration	53.55	73.22	48.25	55.26	37.48	7.63	13.59
Moment propagation	54.10	73.06	48.64	57.24	37.48	7.36	12.22
Moment propagation, no fallback	44.44	70.89	38.98	48.71	19.17	1.62	15.97
Proportional buffer $c\,h$	54.23	73.60	48.25	57.17	37.91	7.13	13.72
Constant 60 s buffer	49.63	69.46	43.72	51.68	33.66	6.52	14.75
Nominal model	48.78	66.73	44.89	49.79	33.71	11.22	13.00
Conventional (event-only)	47.40	65.13	43.54	48.48	32.46	10.28	9.04

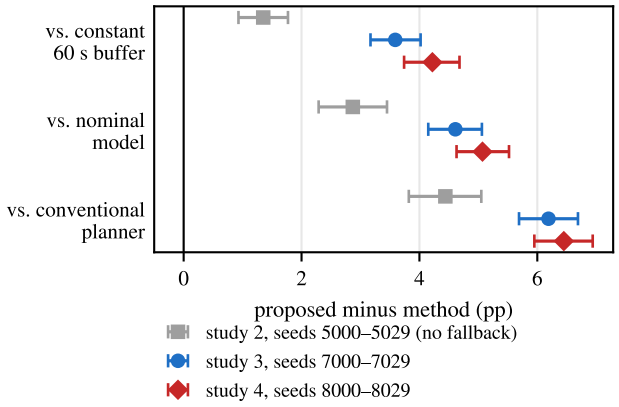


FIGURE 4. Replication on three independent sets of held-out tier B seeds: utility of the proposed replanner minus each method, pooled over twelve configurations, with 95% paired bootstrap intervals over 270 missions per study. Study 2 used the replanner without fallback.

than moment propagation (13.35 against 12.22). The fallback carried over to moment propagation. Without it, moment propagation served 9.66 pp less (9.04 to 10.29, H11) and, at 44.44%, the lowest utility of all models, below the nominal model. A hard chance constraint on every task, however it is evaluated, thus gives up tasks that are still worth attempting. Among the secondary comparisons, online calibration served 0.30 pp less than the model with known disturbance level, the learned stopping rule added 0.34 pp, and the error model ranked the served tasks by risk, with 76.9% served on time under high disturbances against a predicted 71.3% (Supplementary Fig. S9).

E. SOFTWARE-IN-THE-LOOP FLIGHTS

In study 2, 120 held-out flights with mid disturbances and the original windows did not confirm the advantage of the replanner without fallback (H4; −0.25 [−4.02, +3.57] pp to ALNS with 40 ms, Holm-adjusted $p = 0.942$; Supplementary Section S5.2). Tier B predicted the utility of a flight closely ($r = 0.86$), but also expected only +2.82 [−1.20, +6.62] pp on these missions, too little to be confirmed by about 118 missions per method. Study 4C was therefore placed in the regime where tier B predicted the largest effect. It flew 108 flights on 36 new held-out missions with the complete replanner, with and without fallback, and ALNS with 40 ms on the nominal model. Every flight ran within the valid range of the real-time factor, with a mean of 1.96, and the interim analysis did not cross the O'Brien–Fleming bound.

The fallback carried over to the flights. With it, the replanner served 10.35 pp more reward than without it (95% CI 6.41 to 14.39, Holm-adjusted $p < 0.001$, H15), that is, 28.99% against 18.64%, or 56% more, and it was better in 28 of the 36 missions. Against the nominal model, it served 0.57 pp less (−3.49 to 2.29, $p = 0.699$), so H14 is not supported. Tier B predicted both outcomes (Fig. 5). On the same missions, its counterpart gave 0.22 pp less than the nominal model and 8.89 pp more than the model without fallback, with a correlation of $r = 0.83$ over 108 flights. The advantage over the nominal model that sized the study, 5.52 pp on development seeds, did not appear on the held-out missions even in tier B, so its absence in tier C reflects the sampled missions rather than the transfer to the autopilot. Over the full tier B grid, the advantage in this regime was 3.56 pp, which would need about 51 missions per method to be confirmed in tier C.

F. STUDY 5: THE COST OF A LATE ARRIVAL

The utility of studies 1–4 counts only tasks served on time, so a late service earns nothing and costs nothing. Scored again with a cost of λ mean task rewards for every late arrival, in an exploratory analysis without new runs (Supplementary Fig. S10), the advantage of the proposed model over the nominal model grew from 5.07 pp at $\lambda = 0$ to 14.19 pp at $\lambda = 2$. The fallback behaved in the opposite way. Its advantage vanished at a late cost λ^* of 0.5 in study 3, 1.1 for moment propagation in study 4 and 1.5 in tier C. The nominal fallback thus pays off only if a late arrival costs less than about one task reward. Study 5 tests the cost-aware fallback of Section IV-E on 270 new held-out missions (55 080 runs, Table 6, Fig. 6). For $\lambda = 0$, it is the nominal fallback by construction.

All four primary hypotheses were supported. At $\lambda = 1$, the cost-aware fallback reached a score of 45.81% against 43.40% for the nominal fallback, 2.42 pp more (95% CI 1.81

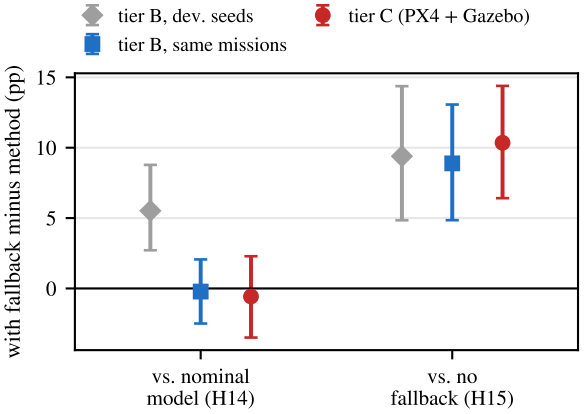


FIGURE 5. Study 4C, held-out seeds 9000–9011, high disturbances with halved windows: utility of the proposed replanner with fallback minus ALNS with 40 ms on the nominal model and minus the replanner without fallback, with 95% paired bootstrap intervals. Tier C (36 missions, PX4 and Gazebo) is shown with its tier B counterpart on the same missions and with the tier B estimate on development seeds from which the size of the study was computed.

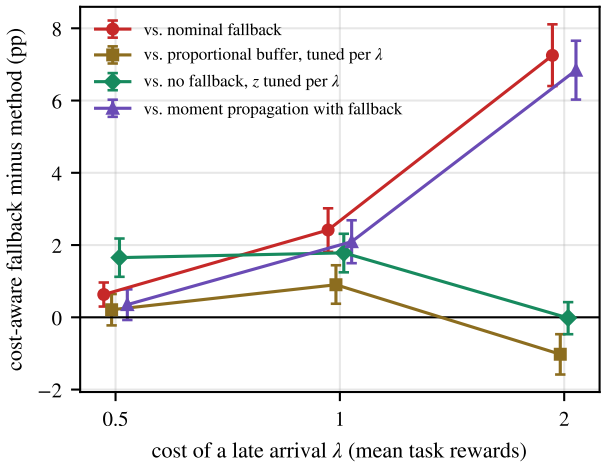


FIGURE 6. Study 5, tier B, held-out seeds 10000–10029: score u_λ of the cost-aware fallback minus each method, when a late arrival costs λ mean task rewards. The cost-aware fallback and the tuned competitors use the λ of the score. 95% paired bootstrap intervals over 270 missions.

to 3.02, Holm-adjusted $p < 0.001$, H16), and served 33% fewer tasks late. Against the proportional buffer tuned for $\lambda = 1$, non-inferiority was shown, and in the fixed sequence it was better by 0.90 pp (0.38 to 1.44, $p < 0.001$, H17). Against the model without fallback with z tuned for $\lambda = 1$, it was better by 1.78 pp (1.25 to 2.31, $p < 0.001$, H18), and over the nominal model by 11.75 pp (H19). Both tuned competitors had been selected on development missions from sixteen and five candidates for the late cost of the score, whereas the cost-aware fallback used the z of study 4. The advantage came from high disturbances, where it served 2.10 and 3.49 pp more than the two tuned competitors. Under mid disturbances, the methods were on par, and with the original windows both tuned competitors were better, by 2.08 and 1.68 pp.

In practical units, per 100 tasks the cost-aware fallback served 50.2 on time and 7.3 late, the nominal fallback 51.3 and 10.9, and the nominal model 48.3 and 15.6. A late arrival was thus 13.8% of all arrivals, against 18.9% and 25.7%, and the fleet flew 92.2 km per mission instead of 101.5 and 104.5 km, consistent with fewer flights to tasks that were likely to be reached too late.

At other late costs, the picture was mixed (Fig. 6). At $\lambda = 0.5$, the cost-aware fallback was on par with the tuned buffer (-0.22 to 0.64 pp) and better than the tuned model without fallback and the nominal fallback. At $\lambda = 2$, it stayed 7.25 pp above the nominal fallback and matched the tuned model without fallback, but the proportional buffer tuned for $\lambda = 2$ was better by 1.02 pp (-1.58 to -0.47), and moment propagation without fallback by 1.10 pp. At this cost, a fallback task needs a probability of at least 0.67 of meeting its deadline, and the rule ignores the delay that it causes to the tasks after it on the same route, which weighs more when lateness is expensive. The competitors, however, have to be tuned again for every late cost. Tuned for $\lambda = 1$, the proportional buffer stayed 1.05 pp behind the cost-aware fallback at $\lambda = 2$ and 1.46 pp behind at $\lambda = 0.5$. A wrong late cost also costs the cost-aware fallback, 1.22 pp when it was run with $\lambda = 1$ and scored at $\lambda = 2$. When a late arrival costs nothing, the rule run with $\lambda = 1$ served 1.20 pp less utility than the nominal fallback.

G. STUDY 6: A PUBLISHED ALLOCATOR AND THE GENERALITY OF THE COST-AWARE FALLBACK

Study 6 comprises 29 160 tier B runs on 270 new held-out missions (Table 7, Fig. 7). Every method with the cost-aware fallback was run with $\lambda = 1$, and the scores for other late costs are computed from the same runs. All four primary hypotheses were supported, each with a Holm-adjusted $p < 0.001$. With the score u_1, the proposed replanner exceeded CBBA by 12.77 pp (95% CI 11.91 to 13.63, H20), and CBBA itself gained 12.12 pp (11.31 to 12.93, H21) when it planned on the deadlines of the cost-aware fallback. Within moment propagation, the cost-aware fallback scored 2.71 pp above the nominal fallback (2.13 to 3.27, H22) and 3.30 pp above the model without fallback (2.83 to 3.79, H23). The advantage of the cost-aware over the nominal fallback in the proposed model, 2.42 pp (1.91 to 2.92), replicates H16 on a second set of seeds.

The difference to CBBA lies in the planning model and not in the search or in the cost of computation. CBBA converged after 2.8 consensus rounds and 5.1 ms per call on average, against 40 ms for ALNS, and on the nominal model it reached almost the utility of ALNS (-1.48 to -0.51 pp). On utility alone ($\lambda = 0$), the proposed replanner served 4.86 pp, or 10.0%, more than CBBA, and 6.72 pp more than CBBA with the event trigger. CBBA served 10.57 tasks late per mission, the proposed replanner 4.91. On the proposed model, CBBA came within 0.65 pp of the proposed replanner at $\lambda = 1$,

TABLE 6. Study 5, tier B held-out results: mean score u_λ (%) for a late arrival that costs λ mean task rewards ($\lambda = 0$ is the utility of studies 2–4), and tasks served late per mission at $\lambda = 1$. The cost-aware fallback and the tuned competitors use the late cost of the column. 270 missions, 12 configurations each.

Method	$\lambda = 0$	0.5	1	2	Late
Cost-aware fallback (proposed)	54.33	49.49	45.81	39.71	4.96
Nominal fallback (study 4)	54.33	48.86	43.40	32.46	7.39
Proportional buffer, tuned per λ	–	49.28	44.91	40.73	4.36
No fallback, z tuned per λ	–	47.84	44.03	39.73	3.00
Moment propagation, fallback	54.57	49.14	43.72	32.87	7.29
Moment propagation, no fallback	45.06	44.00	42.94	40.81	1.50
Constant 60 s buffer	50.33	45.66	40.98	31.64	6.49
Nominal model	49.63	41.85	34.07	18.50	11.08
Conventional (event-only)	47.84	40.56	33.27	18.71	10.25

TABLE 7. Study 6, tier B held-out results: mean score u_λ (%) for a late arrival that costs λ mean task rewards ($\lambda = 0$ is the utility), tasks served late per mission and planning time per call. Methods with the cost-aware fallback use $\lambda = 1$. 270 missions, 12 configurations each.

Method	$\lambda = 0$	0.5	1	2	Late	ms
Proposed (ALNS, cost-aware fallback)	53.42	49.81	46.21	39.01	4.91	40.2
CBBA on the proposed model	51.82	48.69	45.56	39.30	4.25	2.1
CBBA, periodic	48.56	41.00	33.44	18.32	10.57	5.1
CBBA, event-triggered	46.69	39.58	32.46	18.23	9.77	13.2
Moment propagation, cost-aware fallback	52.01	49.49	46.98	41.95	3.52	40.3
Moment propagation, nominal fallback	54.97	49.62	44.27	33.57	7.18	40.2
Moment propagation, no fallback	45.75	44.71	43.67	41.60	1.51	40.2
Nominal model (ALNS)	49.55	41.76	33.98	18.42	11.02	40.1

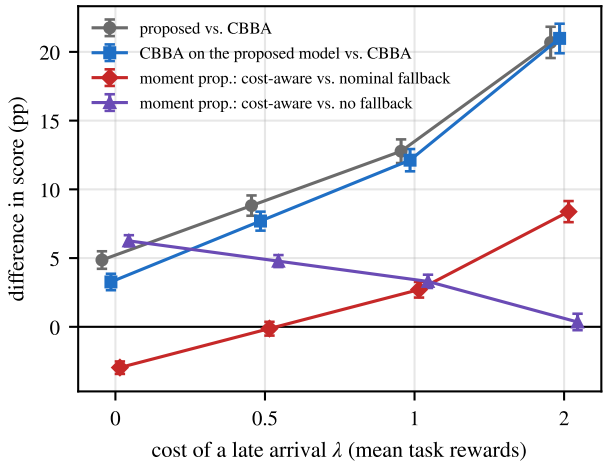


FIGURE 7. Study 6, tier B, held-out seeds 12000–12029: differences in the score u_λ between methods, with 95% paired bootstrap intervals over 270 missions. All methods with the cost-aware fallback were run with $\lambda = 1$ and are scored at every λ from the same runs.

with 2.1 ms per call, so most of the gain does not depend on ALNS. The exception was the regime of high disturbances with halved windows at $\lambda = 0$, where the proposed replanner and CBBA were on par (-0.24 [-1.29, $+0.82$] pp), because a fallback for $\lambda = 1$ gives up tasks that a late arrival would not penalise at $\lambda = 0$.

Within moment propagation, the cost-aware fallback behaved as in the closed form. Without a late cost it lost 2.96 pp against the nominal fallback, it broke even at $\lambda = 0.5$, and its advantage grew to 8.38 pp at $\lambda = 2$. With the cost-aware fallback, moment propagation scored 0.76 pp more than the closed-form model at $\lambda = 1$ (closed form minus moment propagation -1.31 to -0.24) and 2.94 pp more at $\lambda = 2$, whereas the closed form was 1.41 pp better at $\lambda = 0$. Once lateness has a cost, the variance that moment propagation tracks leg by leg therefore pays off, even though it ran fewer search iterations per call (433.2 against 467.4).

VII. DISCUSSION AND LIMITATIONS

The six studies answer the four research questions as follows. On RQ1, the spread of the finish-time error grows almost linearly with the lead time, and a chance constraint built on it reduces to one effective deadline per task that fast heuristics use unchanged. On RQ2, the planning model matters most, the trigger second and the stopping rule least. On RQ3, the replanner served more reward than a constant buffer, the nominal model, a conventional planner and CBBA, and as much as a tuned proportional buffer and moment propagation, but software-in-the-loop flights confirmed only the fallback. On RQ4, the probability of meeting the deadline decided which given-up tasks to attempt without tuning, and this beat competitors tuned for the cost of lateness at one task reward, though not at twice that cost.

The order of priorities holds across the designs. A margin that grows with the lead time raised the utility by more than any choice of stopping rule or trigger, on three independent sets of held-out seeds, without search and with CBBA in place of ALNS. Periodic replanning helped every method on the nominal model, but only by 0.22 pp on the chance-

constrained model, which suggests that both correct the same accumulated deviations, one before and one after they occur. The stopping rule was equivalent to a matched fixed budget in study 2 and added 0.34 pp in study 4. Its main value lies in avoiding the losses of long budgets on slow planners, probably because the improvement curve of this ALNS flattens within a few tens of milliseconds.

The shape of the margin matters more than the way it is computed. The constant buffer was clearly worse than every margin that grows with the lead time, whereas the calibrated effective deadline, a tuned proportional buffer and moment propagation were within 0.38 pp of each other in utility. Because the spread grows almost linearly, a margin $c\,h$ with a suitable slope captures most of it, and under high disturbances the tuned slope nearly equalled the one that the calibrated model implies. Moment propagation evaluates the actual legs of a route and reduced the search rate, but with a cost of lateness its detail paid off in study 6, by 0.76 pp at one task reward and 2.94 pp at two. A planner with a tight budget or a large fleet may therefore prefer the closed form, and a planner with a generous budget and costly lateness the more detailed model. The comparison with CBBA places the gain in the planning model rather than in the allocation method. On the proposed model, CBBA came within 0.65 pp of the proposed replanner while it planned for 2.1 ms per call, so the effective deadlines can be handed to an auction-based allocator without changing its bidding rule.

The probability of meeting the deadline is where the calibrated model earns its place. A hard chance constraint discards every task whose probability falls below $\Phi(z)$, even when a vehicle has nothing better to do, and study 4 shows that this holds however the constraint is evaluated. The nominal fallback recovers these tasks but serves many of them late. With utility alone, a tuned proportional buffer was as good as the calibrated model. Once a late arrival has a cost, the question is no longer only how large the margin is but which tasks are worth attempting at all, and that requires a probability. The closed form turns the break-even rule of Eq. (6) into another effective deadline, which served more than a buffer and a safety factor tuned for one task reward and stayed ahead of the buffer whenever the buffer had been tuned for a different cost. Applied to moment propagation, the same rule behaved in the same way, so it is not tied to the closed form. At twice the cost, a tuned buffer without fallback was better. The rule prices each attempt on its own and ignores the delay that a fallback task causes to later tasks on its route, and a route-level version that also charges this delay is the natural next step.

The two tiers play different roles. The kinematic tier runs tens of thousands of missions and resolves differences of a few percentage points. The software-in-the-loop tier adds the autopilot, the vehicle dynamics and the timing of a real planning loop, but the cost of a flight limits it to a few dozen missions. In both tier C studies, tier B predicted the utility of a flight closely and anticipated every tier C comparison on the same missions, including those that were not significant. The flights therefore support the transfer of the kinematic results and confirm the fallback, but the advantage of the chance constraint over the nominal model rests on tier B. The development estimate that sized study 4C overstated this advantage, so tier C studies should be sized from a conservative effect.

Several limitations bound these conclusions. The error model is linear in the lead time and fitted per disturbance level, so the level is assumed to be known. Online calibration removes this assumption but served 0.30 pp less reward, and its non-inferiority was not shown. The chance constraint is imposed on every task separately rather than jointly on a route. The fallback, the models of study 4 and the cost-aware fallback were designed after the held-out results of earlier studies and were therefore tested on new seeds. The cost-aware fallback assumes that the cost of a late arrival is known and proportional to the reward of the task, and it was tested in tier B only. CBBA ran centrally with a fully connected network, the most favourable case for its communication, so its robustness to lost links was not part of the comparison. Tier C uses three UAVs and fifteen tasks per mission, and software-in-the-loop simulation reproduces the autopilot and the vehicle dynamics but not the sensors, the radio link or the weather of a field test. The latency factor emulates a slower planning computer instead of measuring one, and because the stopping rules read a wall clock, the mean utility of a method varied by up to 0.12 pp between two runs of the same missions. Finally, all methods share one family of scenarios, and other mission types may change the size of the gains.

VIII. CONCLUSION

This paper proposed LTCC, a multi-UAV replanner that plans against a chance constraint on the finish time of every task. Because disturbances persist over a leg, the spread of the finish-time error grows linearly with the lead time, and with an affine error model the chance constraint reduces exactly to an effective deadline per task, so fast heuristics are reused unchanged. In six pre-registered studies on held-out missions, the margin that grows with the lead time, together with the fallback, carried the gain in utility. The replanner served more than the nominal model, a constant buffer, a conventional planner and, by 10.0%, CBBA, and as much as a tuned proportional buffer and moment propagation. The probability of meeting a deadline, which a buffer lacks, lets the replanner price lateness. With a late arrival that costs one task reward, the cost-aware fallback served 2.42 percentage points more than the nominal fallback and more than a buffer and a safety factor tuned for this cost, without tuning of its own, and it improved moment propagation in the same way. At twice this cost, a tuned buffer was better. Software-in-the-loop flights with PX4 and Gazebo confirmed the fallback and agreed closely with the kinematic tier, but did not resolve the advantage over the nominal model. Future work will charge the delay that a fallback task causes on its route, fly the cost-aware fallback in the software-in-the-loop tier, run it inside a distributed allocator, and test the replanner on embedded

hardware and in field flights.

DATA AND CODE AVAILABILITY

The planner, both simulation tiers, the experiment configurations with their SHA-256 hashes, the pre-registered protocols, all raw results and the scripts that produce every number, table and figure of this paper will be made available in a public repository upon publication.

ACKNOWLEDGMENT

The authors would like to express their sincere appreciation to the Vietnam Aerospace University for supporting this research during the 2026–2027 academic year.

REFERENCES

[1] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[2] V. Pillac, M. Gendreau, C. Guéret, and A. L. Medaglia, "A review of dynamic vehicle routing problems," *European Journal of Operational Research*, vol. 225, no. 1, pp. 1–11, 2013.

[3] H. N. Psaraftis, M. Wen, and C. A. Kontovas, "Dynamic vehicle routing problems: Three decades and counting," *Networks*, vol. 67, no. 1, pp. 3–31, 2016.

[4] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.

[5] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[6] F. Errico, G. Desaulniers, M. Gendreau, W. Rei, and L.-M. Rousseau, "A priori optimization with recourse for the vehicle routing problem with hard time windows and stochastic service times," *European Journal of Operational Research*, vol. 249, no. 1, pp. 55–66, 2016.

[7] A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, and C. Requejo, "The robust vehicle routing problem with time windows," *Computers & Operations Research*, vol. 40, no. 3, pp. 856–866, 2013.

[8] J. F. Ehmke, A. M. Campbell, and T. L. Urban, "Ensuring service levels in routing problems with time windows and stochastic travel times," *European Journal of Operational Research*, vol. 240, no. 2, pp. 539–550, 2015.

[9] S. S. Ponda, L. B. Johnson, and J. P. How, "Distributed chance-constrained task allocation for autonomous multi-agent teams," in *Proceedings of the American Control Conference (ACC)*, 2012, pp. 4528–4533.

[10] Z. Gao, M. Zheng, Y. Mei, A. Zheng, and H. Zhong, "Distributionally robust chance-constrained task assignment for heterogeneous UAVs with time windows under uncertain fuel consumption," *Drones*, vol. 9, no. 9, p. 633, 2025.

[11] T. Dean and M. Boddy, "An analysis of time-dependent planning," in *Proceedings of the Seventh National Conference on Artificial Intelligence (AAAI-88)*. AAAI Press, 1988, pp. 49–54.

[12] S. Russell and E. Wefald, *Do the Right Thing: Studies in Limited Rationality*. Cambridge, MA: MIT Press, 1991.

[13] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI Magazine*, vol. 17, no. 3, pp. 73–83, 1996.

[14] J. Svegliato, K. H. Wray, and S. Zilberstein, "Meta-level control of anytime algorithms with online performance prediction," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-18)*. IJCAI Organization, 2018, pp. 1499–1505.

[15] D. Taş, N. Dellaert, T. van Woensel, and T. de Kok, "Vehicle routing problem with stochastic travel times including soft time windows and service costs," *Computers & Operations Research*, vol. 40, no. 1, pp. 214–224, 2013.

[16] M. Boddy and T. L. Dean, "Deliberation scheduling for problem solving in time-constrained environments," *Artificial Intelligence*, vol. 67, no. 2, pp. 245–285, 1994.

[17] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 6235–6240.

[18] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.

[19] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[20] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.

[21] P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research*, vol. 209, no. 1, pp. 1–10, 2011.

[22] S. Wang, W. Sun, and M. Huang, "An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows," *Computers & Industrial Engineering*, vol. 191, p. 110122, 2024.

[23] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 231–247, 2022.

[24] H. Huang, Z. Jiang, T. Yan, and Y. Bai, "Dynamic task allocation for heterogeneous multi-UAVs in uncertain environments based on 4DI-GWO algorithm," *Drones*, vol. 8, no. 6, p. 236, 2024.

[25] C. Qian and B. Fu, "Uncertainty-gated distributed dynamic task allocation for multi-UAV systems under intermittent communication," *Drones*, vol. 10, no. 10, p. 782, 2026.

[26] S.-Y. Hwang, S. Oh, J.-C. Lee, S. Lee, J.-H. Lee, and H.-J. Hwang, "Assignment-preserving replanning for dynamic task allocation in multi-UAV systems," *Applied Sciences*, vol. 16, no. 19, p. 9414, 2026.

[27] Y. Shen, T. Hu, Y. Wan, H. Sun, and M. Tan, "Edge-responsive local-repair particle swarm optimization for dynamic spatiotemporal task allocation of IoT-connected UAV swarms," *IEEE Access*, vol. 14, pp. 115 169–115 190, 2026.

[28] Y. Cong, Y. Li, X. Zhu, and Z. Wang, "An event-triggered path-based bid assignment method for energy-aware dynamic multi-UAV task allocation," *Drones*, vol. 10, no. 9, p. 684, 2026.

[29] G. E. Vieira, J. W. Herrmann, and E. Lin, "Rescheduling manufacturing systems: A framework of strategies, policies, and methods," *Journal of Scheduling*, vol. 6, no. 1, pp. 39–62, 2003.

[30] D. Ouelhadj and S. Petrovic, "A survey of dynamic scheduling in manufacturing systems," *Journal of Scheduling*, vol. 12, no. 4, pp. 417–431, 2009.

[31] Á. Calvo and J. Capitán, "Heterogeneous multirobot task allocation for long-endurance missions in dynamic scenarios," *IEEE Transactions on Robotics*, vol. 41, pp. 6494–6513, 2025.

[32] M. A. Ali, U. Athar, and A. Maqsood, "SITL validation of federated reinforcement learning for priority-based dynamic task allocation in UAV swarms," *Expert Systems*, vol. 43, no. 12, p. e70459, 2026.

[33] L. Zhou and P. Tokekar, "Risk-aware submodular optimization for multirobot coordination," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3064–3084, 2022.

[34] J. Wang, G. Jia, Z. Guo, and Z. Hou, "Impact of uncertain flight time on heterogeneous UAVs' task planning with temporal constraints," *Aerospace*, vol. 11, no. 3, p. 214, 2024.

[35] M. Gendreau, G. Laporte, and R. Séguin, "Stochastic vehicle routing," *European Journal of Operational Research*, vol. 88, no. 1, pp. 3–12, 1996.

[36] R. W. Bent and P. Van Hentenryck, "Scenario-based planning for partially dynamic vehicle routing with stochastic customers," *Operations Research*, vol. 52, no. 6, pp. 977–987, 2004.

[37] X. Li, P. Tian, and S. C. H. Leung, "Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm," *International Journal of Production Economics*, vol. 125, no. 1, pp. 137–145, 2010.

[38] C. E. Clark, "The greatest of a finite set of random variables," *Operations Research*, vol. 9, no. 2, pp. 145–162, 1961.

[39] E. A. Hansen and S. Zilberstein, "Monitoring and control of anytime algorithms: A dynamic programming approach," *Artificial Intelligence*, vol. 126, no. 1–2, pp. 139–157, 2001.

[40] Y. Sung, L. P. Kaelbling, and T. Lozano-Pérez, "Learning when to quit: Meta-reasoning for motion planning," in *Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 4692–4699.

[41] M. Cashmore, A. Coles, B. Cserna, E. Karpas, D. Magazzeni, and W. Ruml, "Temporal planning while the clock ticks," in *Proceedings of the Inter-*

national Conference on Automated Planning and Scheduling (ICAPS), vol. 28. AAAI Press, 2018, pp. 39–46.
[42] S. S. Shperberg, A. Coles, E. Karpas, W. Ruml, and S. E. Shimony, "Situated temporal planning using deadline-aware metareasoning," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31. AAAI Press, 2021, pp. 340–348.
[43] A. Dionne, J. Thayer, and W. Ruml, "Deadline-aware search using on-line measures of behavior," in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, vol. 2, no. 1. AAAI Press, 2011, pp. 39–46.
[44] M. W. P. Savelsbergh, "The vehicle routing problem with time windows: Minimizing route duration," *ORSA Journal on Computing*, vol. 4, no. 2, pp. 146–154, 1992.
[45] J.-Y. Potvin and J.-M. Rousseau, "A parallel route building algorithm for the vehicle routing and scheduling problem with time windows," *European Journal of Operational Research*, vol. 66, no. 3, pp. 331–340, 1993.
[46] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming – CP98*, ser. Lecture Notes in Computer Science, vol. 1520. Springer, 1998, pp. 417–431.
[47] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
[48] P. McCullagh and J. A. Nelder, *Generalized Linear Models*, 2nd ed. London: Chapman and Hall, 1989.
[49] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York: McGraw-Hill Education, 2015.
[50] L. B. Johnson, H.-L. Choi, S. S. Ponda, and J. P. How, "Allowing non-submodular score functions in distributed task allocation," in *Proceedings of the 51st IEEE Conference on Decision and Control (CDC)*, 2012, pp. 4702–4708.
[51] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman and Hall, 1993.
[52] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.



CAO-PHUC HA is currently pursuing the B.S. degree in Artificial Intelligence at FPT University, Vietnam. His research interests include artificial intelligence and robotics, with applications to autonomous vehicles, drones and unmanned aerial systems.

ANH NGUYEN VU MINH is with the Department of Artificial Intelligence, FPT University, Ho Chi Minh City, Vietnam.



PHU-NGUYEN LE received the B.S. degree in electrical engineering from Da Nang University of Technology, Vietnam, in 2012, and the Ph.D. degree from the University of Ulsan, South Korea, in 2021. He is currently with the Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University. His research interests include sensor-based robotic applications, robot calibration and AI-based robotic control.